

Projet B - Séquence 4.5 - Script de création de la base de données

Script SQL Final pour Supabase

Voici le script prêt à être exécuté dans Supabase (via l'onglet **SQL Editor**).

Voici le **script SQL complet et optimisé** pour **StudyFlow AI**, intégrant toutes les corrections, ajouts et optimisations identifiés dans les documents "**Projet B - Séquence 4.3**" et "**Projet B - Séquence 4.4**", ainsi que votre MCD initial.

Ce script est prêt à être exécuté dans **Supabase (PostgreSQL)**.



Script SQL Final pour StudyFlow AI

```
-- =====
-- 1. ACTIVATION DES EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- =====
-- 2. CRÉATION DES TYPES ENUM
-- =====
CREATE TYPE role_utilisateur AS ENUM ('étudiant', 'professeur', 'admin');
CREATE TYPE statut_utilisateur AS ENUM ('actif', 'inactif', 'suspendu');
CREATE TYPE type_ressource AS ENUM ('Document', 'Video', 'Audio', 'Lien');
CREATE TYPE statut_tache AS ENUM ('planifiée', 'à faire', 'en cours', 'terminée');
CREATE TYPE priorite_tache AS ENUM ('basse', 'moyenne', 'haute');
CREATE TYPE type_calendrier AS ENUM ('personnel', 'cours', 'examens', 'groupe');
CREATE TYPE statut_notification AS ENUM ('non lue', 'lue', 'archivée');
CREATE TYPE type_abonnement AS ENUM ('gratuit', 'étudiant', 'professeur', 'institutionnel');
CREATE TYPE statut_plan_revision AS ENUM ('actif', 'terminé', 'annulé');
CREATE TYPE statut_session_revision AS ENUM ('planifiée', 'en cours', 'terminée');
CREATE TYPE type_question_quiz AS ENUM ('QCM', 'texte libre', 'vrai/faux');
CREATE TYPE type_donnees_analytiques AS ENUM ('temps_étude', 'score_quiz', 'connexion');
CREATE TYPE statut_cours AS ENUM ('actif', 'archivé');
CREATE TYPE statut_groupe AS ENUM ('actif', 'archivé');
CREATE TYPE statut_abonnement AS ENUM ('actif', 'expiré', 'annulé');
CREATE TYPE role_groupe AS ENUM ('admin', 'membre');
CREATE TYPE type_evenement AS ENUM ('tâche', 'échéance', 'rappel');
CREATE TYPE type_contexte_notification AS ENUM ('tâche', 'échéance', 'message', 'rappel');
```

```

CREATE TYPE visibilite_ressource AS ENUM ('publique', 'privée', 'groupe');

-- =====
-- 3. TABLES PRINCIPALES
-- =====

-- Table Établissement
CREATE TABLE etablissement (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  adresse TEXT,
  ville VARCHAR(100),
  pays VARCHAR(100),
  code_postal VARCHAR(20),
  logo BYTEA,
  date_creation TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Utilisateur
CREATE TABLE utilisateur (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(100) NOT NULL,
  prénom VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  mot_de_passe TEXT NOT NULL,
  rôle role_utilisateur NOT NULL DEFAULT 'étudiant',
  date_inscription TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  dernière_connexion TIMESTAMP WITH TIME ZONE,
  photo_profil BYTEA,
  statut statut_utilisateur NOT NULL DEFAULT 'actif',
  email_verifié BOOLEAN DEFAULT FALSE,
  jeton_reinitialisation TEXT,
  id_établissement UUID REFERENCES etablissement(id) ON DELETE SET NULL
);

-- Table Coursus
CREATE TABLE cursus (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  description TEXT,
  id_établissement UUID NOT NULL REFERENCES etablissement(id) ON DELETE CASCADE,
  statut statut_cours NOT NULL DEFAULT 'actif'
);

-- Table Cours
CREATE TABLE cours (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  description TEXT,
  id_professeur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
  id_cursus UUID NOT NULL REFERENCES cursus(id) ON DELETE CASCADE,
  date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  date_mise_à_jour TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  statut statut_cours NOT NULL DEFAULT 'actif',

```

```

    matière VARCHAR(100)
);

-- Table Utilisateur_Cours (N-à-N)
CREATE TABLE utilisateur_cours (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    utilisateur_id UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    cours_id UUID NOT NULL REFERENCES cours(id) ON DELETE CASCADE,
    rôle role_utilisateur NOT NULL,
    date_adhésion TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE (utilisateur_id, cours_id)
);

-- Table Ressource
CREATE TABLE ressource (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    titre VARCHAR(255) NOT NULL,
    type type_ressource NOT NULL,
    contenu_texte TEXT,
    contenu_url TEXT,
    id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    note_moyenne FLOAT DEFAULT 0,
    visibilité visibilite_ressource NOT NULL DEFAULT 'privée',
    CHECK (
        (type = 'Document' OR type = 'Audio') AND contenu_texte IS NOT NULL AND contenu_url IS NULL
        OR
        (type = 'Video' OR type = 'Lien') AND contenu_url IS NOT NULL AND contenu_texte IS NULL
    )
);

-- Table Tâche
CREATE TABLE tâche (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    date_limite TIMESTAMP WITH TIME ZONE NOT NULL,
    statut statut_tache NOT NULL DEFAULT 'planifiée',
    priorité priorite_tache NOT NULL DEFAULT 'moyenne',
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
    id_groupe UUID REFERENCES groupe_collaboratif(id) ON DELETE CASCADE,
    calendrier_id UUID REFERENCES calendrier(id) ON DELETE SET NULL,
    CHECK (id_cours IS NULL OR id_groupe IS NULL)
);

-- Table Échéance
CREATE TABLE échéance (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    description TEXT,
    date_limite TIMESTAMP WITH TIME ZONE NOT NULL,
    id_tâche UUID NOT NULL REFERENCES tâche(id) ON DELETE CASCADE
);

```

-- Table Événement

```
CREATE TABLE événement (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  titre VARCHAR(255) NOT NULL,  
  description TEXT,  
  date_debut TIMESTAMP WITH TIME ZONE NOT NULL,  
  date_fin TIMESTAMP WITH TIME ZONE NOT NULL,  
  calendrier_id UUID NOT NULL REFERENCES calendrier(id) ON DELETE CASCADE,  
  couleur VARCHAR(50),  
  type type_evenement NOT NULL,  
  id_tâche UUID REFERENCES tâche(id) ON DELETE CASCADE,  
  id_échéance UUID REFERENCES échéance(id) ON DELETE CASCADE  
);
```

-- Table Calendrier

```
CREATE TABLE calendrier (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  titre VARCHAR(255) NOT NULL,  
  description TEXT,  
  date_debut TIMESTAMP WITH TIME ZONE NOT NULL,  
  date_fin TIMESTAMP WITH TIME ZONE NOT NULL,  
  type type_calendrier NOT NULL  
);
```

-- Table Groupe_Collaboratif

```
CREATE TABLE groupe_collaboratif (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  nom VARCHAR(255) NOT NULL,  
  description TEXT,  
  id_créateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  date_création TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  statut statut_groupe NOT NULL DEFAULT 'actif'  
);
```

-- Table Membre_Groupe

```
CREATE TABLE membre_groupe (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  id_groupe UUID NOT NULL REFERENCES groupe_collaboratif(id) ON DELETE CASCADE,  
  date_adhésion TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  rôle role_groupe NOT NULL,  
  UNIQUE (id_utilisateur, id_groupe)  
);
```

-- Table Notification

```
CREATE TABLE notification (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,  
  titre VARCHAR(255) NOT NULL,  
  message TEXT NOT NULL,  
  date_envoi TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  statut statut_notification NOT NULL DEFAULT 'non lue',
```

```

type type_contexte_notification NOT NULL,
id_tâche UUID REFERENCES tâche(id) ON DELETE CASCADE,
id_échéance UUID REFERENCES échéance(id) ON DELETE CASCADE,
id_message UUID,
CHECK (
    (type = 'tâche' AND id_tâche IS NOT NULL AND id_échéance IS NULL AND id_message IS NULL)
    OR
    (type = 'échéance' AND id_échéance IS NOT NULL AND id_tâche IS NULL AND id_message IS
NULL)
    OR
    (type = 'message' AND id_message IS NOT NULL AND id_tâche IS NULL AND id_échéance IS
NULL)
    OR
    (type = 'rappel' AND id_tâche IS NULL AND id_échéance IS NULL AND id_message IS NULL)
)
);

```

-- Table Abonnement

```

CREATE TABLE abonnement (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    type type_abonnement NOT NULL,
    date_debut TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    date_fin TIMESTAMP WITH TIME ZONE,
    statut statut_abonnement NOT NULL DEFAULT 'actif'
);

```

-- Table Plan_Révision

```

CREATE TABLE plan_revision (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    date_debut TIMESTAMP WITH TIME ZONE NOT NULL,
    date_fin TIMESTAMP WITH TIME ZONE NOT NULL,
    statut statut_plan_revision NOT NULL DEFAULT 'actif'
);

```

-- Table Session_Révision

```

CREATE TABLE session_revision (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_plan_revision UUID NOT NULL REFERENCES plan_revision(id) ON DELETE CASCADE,
    date TIMESTAMP WITH TIME ZONE NOT NULL,
    durée INT NOT NULL,
    statut statut_session_revision NOT NULL DEFAULT 'planifiée'
);

```

-- Table Quiz

```

CREATE TABLE quiz (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    id_cours UUID REFERENCES cours(id) ON DELETE CASCADE,
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,

```

```

    date_creation TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Question_Quiz
CREATE TABLE question_quiz (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_quiz UUID NOT NULL REFERENCES quiz(id) ON DELETE CASCADE,
    contenu TEXT NOT NULL,
    type type_question_quiz NOT NULL,
    points INT NOT NULL DEFAULT 1
);

-- Table Réponse_Quiz
CREATE TABLE reponse_quiz (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_question_quiz UUID NOT NULL REFERENCES question_quiz(id) ON DELETE CASCADE,
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    contenu TEXT,
    est_correcte BOOLEAN NOT NULL DEFAULT FALSE,
    date_soumission TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE (id_question_quiz, id_utilisateur)
);

-- Table Progression
CREATE TABLE progression (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    id_tâche UUID REFERENCES tâche(id) ON DELETE CASCADE,
    id_quiz UUID REFERENCES quiz(id) ON DELETE CASCADE,
    note FLOAT CHECK (note BETWEEN 0 AND 20),
    date_realisation TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Données_Analytiques
CREATE TABLE donnees_analytiques (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    type type_donnees_analytiques NOT NULL,
    valeur FLOAT NOT NULL,
    date_enregistrement TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table Note_Ressource (pour calculer note_moyenne)
CREATE TABLE note_ressource (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    id_ressource UUID NOT NULL REFERENCES ressource(id) ON DELETE CASCADE,
    id_utilisateur UUID NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE,
    note FLOAT NOT NULL CHECK (note BETWEEN 0 AND 5),
    date_notation TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE (id_ressource, id_utilisateur)
);

-- =====
-- 4. INDEX POUR L'OPTIMISATION

```

```
-- =====
CREATE INDEX idx_utilisateur_email ON utilisateur(email);
CREATE INDEX idx_utilisateur_etablissement ON utilisateur(id_etablissement);
CREATE INDEX idx_utilisateur_cours_utilisateur ON utilisateur_cours(utilisateur_id);
CREATE INDEX idx_utilisateur_cours_cours ON utilisateur_cours(cours_id);
CREATE INDEX idx_cours_professeur ON cours(id_professeur);
CREATE INDEX idx_cours_cursus ON cours(id_cursus);
CREATE INDEX idx_tache_utilisateur ON tache(id_utilisateur);
CREATE INDEX idx_tache_cours ON tache(id_cours);
CREATE INDEX idx_tache_groupe ON tache(id_groupe);
CREATE INDEX idx_tache_statut ON tache(statut);
CREATE INDEX idx_tache_priorite ON tache(priorite);
CREATE INDEX idx_tache_date_limite ON tache(date_limite);
CREATE INDEX idx_echeance_tache ON echeance(id_tache);
CREATE INDEX idx_echeance_date_limite ON echeance(date_limite);
CREATE INDEX idx_evenement_calendrier ON evenement(calendrier_id);
CREATE INDEX idx_evenement_date_debut ON evenement(date_debut);
CREATE INDEX idx_ressource_cours ON ressource(id_cours);
CREATE INDEX idx_ressource_utilisateur ON ressource(id_utilisateur);
CREATE INDEX idx_ressource_type ON ressource(type);
CREATE INDEX idx_groupe_createur ON groupe_collaboratif(id_createur);
CREATE INDEX idx_membre_groupe_utilisateur ON membre_groupe(id_utilisateur);
CREATE INDEX idx_membre_groupe_groupe ON membre_groupe(id_groupe);
CREATE INDEX idx_notification_utilisateur ON notification(id_utilisateur);
CREATE INDEX idx_notification_statut ON notification(statut);
CREATE INDEX idx_quiz_cours ON quiz(id_cours);
CREATE INDEX idx_quiz_utilisateur ON quiz(id_utilisateur);
CREATE INDEX idx_reponse_quiz_question ON reponse_quiz(id_question_quiz);
CREATE INDEX idx_reponse_quiz_utilisateur ON reponse_quiz(id_utilisateur);
CREATE INDEX idx_progression_utilisateur ON progression(id_utilisateur);
CREATE INDEX idx_progression_tache ON progression(id_tache);
CREATE INDEX idx_progression_quiz ON progression(id_quiz);
```

```
-- =====
-- 5. TRIGGERS POUR LA SYNCHRONISATION ET LA COHERENCE
-- =====
```

```
-- Trigger pour mettre à jour dernière_connexion
CREATE OR REPLACE FUNCTION update_derniere_connexion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.derniere_connexion = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_update_derniere_connexion
BEFORE UPDATE ON utilisateur
FOR EACH ROW
EXECUTE FUNCTION update_derniere_connexion();
```

```
-- Trigger pour synchroniser Tâche → Événement
CREATE OR REPLACE FUNCTION sync_tache_evenement()
RETURNS TRIGGER AS $$
```

```

BEGIN
IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
  -- Créer ou mettre à jour un événement pour la tâche
  INSERT INTO événement (
    titre,
    description,
    date_debut,
    date_fin,
    calendrier_id,
    couleur,
    type,
    id_tâche
  )
  VALUES (
    NEW.titre,
    NEW.description,
    NEW.date_limite,
    NEW.date_limite + INTERVAL '1 hour',
    NEW.calendrier_id,
    '#FF5733',
    'tâche',
    NEW.id
  )
  ON CONFLICT (id_tâche)
  DO UPDATE SET
    titre = EXCLUDED.titre,
    description = EXCLUDED.description,
    date_debut = EXCLUDED.date_debut,
    date_fin = EXCLUDED.date_fin;

  -- Synchroniser les échéances de la tâche
  INSERT INTO événement (
    titre,
    description,
    date_debut,
    date_fin,
    calendrier_id,
    couleur,
    type,
    id_échéance
  )
  SELECT
    e.description,
    e.description,
    e.date_limite,
    e.date_limite + INTERVAL '1 hour',
    NEW.calendrier_id,
    '#33FF57',
    'échéance',
    e.id
  FROM échéance e
  WHERE e.id_tâche = NEW.id
  ON CONFLICT (id_échéance)
  DO UPDATE SET

```



```

        titre = EXCLUDED.titre,
        description = EXCLUDED.description,
        date_debut = EXCLUDED.date_debut,
        date_fin = EXCLUDED.date_fin;
ELSIF TG_OP = 'DELETE' THEN
    -- Supprimer les événements liés à la tâche
    DELETE FROM événement WHERE id_tâche = OLD.id;
    -- Supprimer les événements liés aux échéances de la tâche
    DELETE FROM événement WHERE id_échéance IN (SELECT id FROM échéance WHERE
id_tâche = OLD.id);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_sync_tache_evenement
AFTER INSERT OR UPDATE OR DELETE ON tâche
FOR EACH ROW
EXECUTE FUNCTION sync_tache_evenement();

-- Trigger pour synchroniser Échéance → Événement
CREATE OR REPLACE FUNCTION sync_echeance_evenement()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
        INSERT INTO événement (
            titre,
            description,
            date_debut,
            date_fin,
            calendrier_id,
            couleur,
            type,
            id_échéance
        )
        VALUES (
            NEW.description,
            NEW.description,
            NEW.date_limite,
            NEW.date_limite + INTERVAL '1 hour',
            (SELECT calendrier_id FROM tâche WHERE id = NEW.id_tâche),
            '#33FF57',
            'échéance',
            NEW.id
        )
        ON CONFLICT (id_échéance)
        DO UPDATE SET
            titre = EXCLUDED.titre,
            description = EXCLUDED.description,
            date_debut = EXCLUDED.date_debut,
            date_fin = EXCLUDED.date_fin;
    ELSIF TG_OP = 'DELETE' THEN
        DELETE FROM événement WHERE id_échéance = OLD.id;
    END IF;

```

```

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_sync_echeance_evenement
AFTER INSERT OR UPDATE OR DELETE ON échéance
FOR EACH ROW
EXECUTE FUNCTION sync_echeance_evenement();

-- Trigger pour protéger le créateur du groupe
CREATE OR REPLACE FUNCTION protect_createur_groupe()
RETURNS TRIGGER AS $$
DECLARE
    est_createur BOOLEAN;
BEGIN
    IF TG_OP = 'DELETE' THEN
        SELECT groupe_collaboratif.id_créateur = OLD.id_utilisateur INTO est_createur
        FROM groupe_collaboratif
        WHERE groupe_collaboratif.id = OLD.id_groupe;

        IF est_createur THEN
            RAISE EXCEPTION 'Impossible de supprimer le créateur du groupe.';
        END IF;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_protect_createur_groupe
BEFORE DELETE ON membre_groupe
FOR EACH ROW
EXECUTE FUNCTION protect_createur_groupe();

-- Trigger pour recalculer note_moyenne dans Ressource
CREATE OR REPLACE FUNCTION recalculer_note_moyenne()
RETURNS TRIGGER AS $$
DECLARE
    moyenne FLOAT;
BEGIN
    SELECT AVG(note) INTO moyenne
    FROM note_ressource
    WHERE id_ressource = COALESCE(NEW.id_ressource, OLD.id_ressource);

    IF moyenne IS NULL THEN
        moyenne = 0;
    END IF;

    UPDATE ressource
    SET note_moyenne = moyenne
    WHERE id = COALESCE(NEW.id_ressource, OLD.id_ressource);

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trigger_recalculer_note_moyenne_insert
AFTER INSERT ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();
```

```
CREATE TRIGGER trigger_recalculer_note_moyenne_update
AFTER UPDATE ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();
```

```
CREATE TRIGGER trigger_recalculer_note_moyenne_delete
AFTER DELETE ON note_ressource
FOR EACH ROW
EXECUTE FUNCTION recalculer_note_moyenne();
```

```
-- =====
-- 6. VUE SQL POUR LE TABLEAU DE BORD
-- =====
```

```
CREATE OR REPLACE VIEW vue_tableau_de_bord AS
SELECT
    u.id AS utilisateur_id,
    u.nom,
    u.prénom,
    u.email,
    u.rôle,
    -- Tâches
    COUNT(CASE WHEN t.statut = 'à faire' THEN 1 END) AS tâches_à_faire,
    COUNT(CASE WHEN t.statut = 'en cours' THEN 1 END) AS tâches_en_cours,
    COUNT(CASE WHEN t.statut = 'terminée' THEN 1 END) AS tâches_terminées,
    -- Notifications
    COUNT(CASE WHEN n.statut = 'non lue' THEN 1 END) AS notifications_non_lues,
    -- Progression
    COALESCE(AVG(p.note), 0) AS note_moyenne,
    -- Échéances
    MIN(e.date_limite) AS prochaine_échéance,
    -- Calendrier
    COUNT(DISTINCT ev.id) AS événements_à_venir,
    -- Groupes
    COUNT(DISTINCT mg.id_groupe) AS groupes_rejoins,
    -- Ressources
    COUNT(DISTINCT r.id) AS ressources_créées
FROM
    utilisateur u
LEFT JOIN tâche t ON u.id = t.id_utilisateur
LEFT JOIN notification n ON u.id = n.id_utilisateur
LEFT JOIN progression p ON u.id = p.id_utilisateur
LEFT JOIN échéance e ON u.id = (SELECT id_utilisateur FROM tâche WHERE id = e.id_tâche)
LEFT JOIN événement ev ON u.id = (SELECT id_utilisateur FROM calendrier WHERE id =
ev.calendrier_id) AND ev.date_debut > NOW()
LEFT JOIN membre_groupe mg ON u.id = mg.id_utilisateur
LEFT JOIN ressource r ON u.id = r.id_utilisateur
WHERE
    u.statut = 'actif'
```

GROUP BY

u.id, u.nom, u.prénom, u.email, u.rôle;

-- =====

-- 7. RÈGLES DE SÉCURITÉ (RLS)

-- =====

-- Activer RLS sur toutes les tables

```
ALTER TABLE utilisateur ENABLE ROW LEVEL SECURITY;
ALTER TABLE etablissement ENABLE ROW LEVEL SECURITY;
ALTER TABLE cursus ENABLE ROW LEVEL SECURITY;
ALTER TABLE cours ENABLE ROW LEVEL SECURITY;
ALTER TABLE utilisateur_cours ENABLE ROW LEVEL SECURITY;
ALTER TABLE ressource ENABLE ROW LEVEL SECURITY;
ALTER TABLE tâche ENABLE ROW LEVEL SECURITY;
ALTER TABLE échéance ENABLE ROW LEVEL SECURITY;
ALTER TABLE événement ENABLE ROW LEVEL SECURITY;
ALTER TABLE calendrier ENABLE ROW LEVEL SECURITY;
ALTER TABLE groupe_collaboratif ENABLE ROW LEVEL SECURITY;
ALTER TABLE membre_groupe ENABLE ROW LEVEL SECURITY;
ALTER TABLE notification ENABLE ROW LEVEL SECURITY;
ALTER TABLE abonnement ENABLE ROW LEVEL SECURITY;
ALTER TABLE plan_revision ENABLE ROW LEVEL SECURITY;
ALTER TABLE session_revision ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE question_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE reponse_quiz ENABLE ROW LEVEL SECURITY;
ALTER TABLE progression ENABLE ROW LEVEL SECURITY;
ALTER TABLE donnees_analytiques ENABLE ROW LEVEL SECURITY;
ALTER TABLE note_ressource ENABLE ROW LEVEL SECURITY;
```

-- Politiques RLS pour Utilisateur

CREATE POLICY "Les utilisateurs peuvent voir leur propre profil"

ON utilisateur

FOR SELECT

USING (auth.uid() = id::TEXT);

CREATE POLICY "Les utilisateurs peuvent mettre à jour leur propre profil"

ON utilisateur

FOR UPDATE

USING (auth.uid() = id::TEXT);

-- Politiques RLS pour Tâche

CREATE POLICY "Les utilisateurs peuvent voir leurs propres tâches"

ON tâche

FOR SELECT

USING (id_utilisateur = auth.uid()::UUID);

CREATE POLICY "Les utilisateurs peuvent créer leurs propres tâches"

ON tâche

FOR INSERT

WITH CHECK (id_utilisateur = auth.uid()::UUID);

CREATE POLICY "Les utilisateurs peuvent mettre à jour leurs propres tâches"

ON tâche

```

FOR UPDATE
USING (id_utilisateur = auth.uid()::UUID);

-- Politiques RLS pour Ressource
CREATE POLICY "Les utilisateurs peuvent voir leurs propres ressources ou celles partagées"
ON ressource
FOR SELECT
USING (
    id_utilisateur = auth.uid()::UUID
    OR visibilité = 'publique'
    OR EXISTS (
        SELECT 1 FROM utilisateur_cours uc
        WHERE uc.utilisateur_id = auth.uid()::UUID
        AND uc.cours_id = ressource.id_cours
    )
    OR EXISTS (
        SELECT 1 FROM membre_groupe mg
        JOIN groupe_collaboratif g ON mg.id_groupe = g.id
        WHERE mg.id_utilisateur = auth.uid()::UUID
        AND g.id = (SELECT id_groupe FROM tâche WHERE id = ressource.id_cours)
    )
);

-- Politiques RLS pour Groupe_Collaboratif
CREATE POLICY "Les utilisateurs peuvent voir les groupes dont ils sont membres"
ON groupe_collaboratif
FOR SELECT
USING (
    id_créateur = auth.uid()::UUID
    OR EXISTS (
        SELECT 1 FROM membre_groupe
        WHERE id_groupe = groupe_collaboratif.id
        AND id_utilisateur = auth.uid()::UUID
    )
);

-- Politiques RLS pour Notification
CREATE POLICY "Les utilisateurs peuvent voir leurs propres notifications"
ON notification
FOR SELECT
USING (id_utilisateur = auth.uid()::UUID);

-- Politiques RLS pour Calendrier
CREATE POLICY "Les utilisateurs peuvent voir leurs propres calendriers"
ON calendrier
FOR ALL
USING (id_utilisateur = auth.uid()::UUID);

-- Politiques RLS pour Quiz
CREATE POLICY "Les utilisateurs peuvent voir les quiz des cours auxquels ils sont inscrits"
ON quiz
FOR SELECT
USING (
    id_utilisateur = auth.uid()::UUID

```

```
OR EXISTS (  
  SELECT 1 FROM utilisateur_cours uc  
  WHERE uc.utilisateur_id = auth.uid()::UUID  
  AND uc.cours_id = quiz.id_cours  
)  
);
```

Résumé des Éléments Clés

✓ Tables Ajoutées

1. **utilisateur_cours** : Relation N-à-N entre **Utilisateur** et **Cours** avec rôle.
2. **échéance** : Gestion des échéances pour les tâches.
3. **événement** : Synchronisation avec **Tâche** et **Échéance** pour le calendrier.
4. **progression** : Suivi des notes et réalisations.
5. **note_ressource** : Pour calculer la **note_moyenne** des ressources.

✓ Champs Ajoutés/Modifiés

- **Utilisateur** : **email_verifié**, **jeton_reinitialisation**.
- **Cours** : **statut**, **matière**.
- **Ressource** : **contenu_texte**, **contenu_url**, **visibilité** (remplace **contenu**).
- **Tâche** : **calendrier_id**, contrainte **CHECK** pour **cours_id/groupe_id**.
- **Notification** : **type**, contrainte **CHECK** pour le contexte.

✓ Contraintes et Triggers

1. **Synchronisation Tâche/Échéance** → **Événement** : Triggers pour éviter les duplications.
2. **Protection du créateur de groupe** : Trigger pour empêcher sa suppression.
3. **Recalcul de note_moyenne** : Trigger sur **note_ressource**.
4. **Mise à jour de dernière_connexion** : Trigger sur **Utilisateur**.

✓ Vue SQL

- **vue_tableau_de_bord** : Agrège les métriques (tâches, notifications, échéances, etc.).

✅ RLS (Row-Level Security)

- Politiques pour restreindre l'accès aux données par utilisateur.

🚀 Prochaines Étapes

1. **Exécutez ce script** dans **Supabase SQL Editor**.
2. **Testez les données** :
 - Insérez des utilisateurs, cours, tâches, etc.
 - Vérifiez que les triggers fonctionnent (ex : synchronisation **Tâche** → **Événement**).
3. **Intégrez avec Ionic** :
 - Utilisez le client Supabase pour interagir avec la base.

Si vous avez besoin d'ajustements ou d'explications supplémentaires, n'hésitez pas à me le demander ! 😊